

CURSO DE BANCO DE DADOS
PROJETO INTEGRADOR

Projeto 3 — Banco de Dados

Modelagem Híbrida: SQL vs NoSQL
sobre o dataset Northwind Traders

Entrega 01 — Análise de Negócio, Modelo Conceitual,
Plano Híbrido e Modelo Relacional

Aluno
Giovanna Espindola

Professora
Fabiana Rocha

06 de setembro de 2026

Sumário

00 — Definição do Trabalho e Área de Foco	3
1. Área de foco	3
2. O domínio	4
3. A pergunta central	4
4. Escopo	4
5. Como o trabalho está organizado	5
6. Duas regras que o trabalho seguiu	5
01 — Análise de Negócio e Perfilamento do Northwind	6
1. O que foi feito, e como reproduzir	6
2. O processo de negócio, em prosa	6
3. A base em números	7
4. Fraquezas de modelagem encontradas	9
5. O que a base tem de bom	12
6. Perguntas de negócio que a base responde	13
7. Limitações declaradas	13
8. Próximo passo	14
02 — Modelo Conceitual (ER) do Northwind	15
1. O arquivo do diagrama	15
2. As onze entidades, em três blocos	15
3. Justificativa de cada cardinalidade	16
4. As três decisões do modelo, e o que foi descartado	17
5. O que este modelo deliberadamente não mostra	17
6. Próximo passo	18
03 — Plano Híbrido: PostgreSQL e MongoDB no mesmo projeto	19
1. A pergunta que este documento responde	19
2. O que cada banco faz bem — por critério, não por adjetivo	19
3. A arquitetura escolhida	20
4. O desenho do documento — o que muda de um lado para o outro	21
5. Como a comparação vai ser feita	21
6. O que este experimento NÃO prova — declarado antes de medir	21
7. O critério que vai decidir a resposta final	22

8. Próximo passo	22
04 — Modelo Relacional: o schema nw	23
1. Por que existe um segundo schema	23
2. O que o banco passou a impedir	25
3. Normalização, com exemplo do próprio Northwind	27
3.1 Duas cardinalidades que a chave estrangeira não consegue expressar	28
4. Índices: quatro entraram, seis foram recusados	29
5. Quatro views, nenhuma decorativa	30
6. Reproduzir do zero	30
Apêndice A — Validação da carga public → nw	32

00 — Definição do Trabalho e Área de Foco

Entrega 01 · Projeto Integrador Área 03 — Banco de Dados (IPOG) Autor: Giovanne Espíndola · Trabalho individual · Data: 31/08/2026

1. Área de foco

Modelagem e análise de dados de vendas, com foco na escolha de tecnologia de armazenamento — comparando uma solução **relacional** (PostgreSQL 16) com uma **orientada a documentos** (MongoDB 7) sobre o mesmo domínio de negócio.

Por que PostgreSQL, e não MySQL

O enunciado deixa a escolha aberta entre os dois. A escolha foi pelo PostgreSQL, e não por gosto — por cinco recursos que **este trabalho usa de fato**:

Recurso	Onde é usado aqui
DDL transacional — <code>CREATE TABLE</code> dentro de <code>BEGIN ... ROLLBACK</code>	testar constraint e índice sem sujar o banco; o MySQL <i>faz commit</i> implícito em DDL, e o teste não teria volta
Tipo ARRAY	<code>vw_hierarquia_funcionarios</code> guarda o caminho até o topo da hierarquia num array
Cláusula FILTER (<code>WHERE ...</code>) em agregação	todos os relatórios de validação; no MySQL vira <code>SUM(CASE WHEN ... END)</code> , bem menos legível
EXPLAIN (<code>... BUFFERS</code>)	mostra quantas páginas cada plano toca, e foi com ele que metade das recusas de índice foi decidida em vez de opinada
COMMENT ON em índice e schema	os 4 índices e o próprio schema <code>nw</code> carregam a explicação dentro do catálogo; o MySQL só comenta tabela e coluna

Na Entrega 03 entra ainda **GROUPING SETS**, para a pergunta 16 de sazonalidade — o MySQL só tem **WITH ROLLUP**.

Sendo justo com o MySQL, e isso importa numa defesa: as coisas que normalmente se citam contra ele **não valem mais**. Desde a versão 8.0 ele tem CTE recursiva e *window functions*, que dão conta das perguntas 07 a 11 e 15, e desde a 8.0.18 tem **EXPLAIN ANALYZE** — o que o PostgreSQL acrescenta ali é o **BUFFERS**, não a medição em si. A diferença real está nos itens acima, e a mais decisiva para *este* projeto é a primeira: metade das verificações feitas aqui só é possível porque dá para testar uma mudança de schema e desfazê-la.

E o MongoDB, foi escolha?

Não da mesma forma. O enunciado deixa o relacional em aberto ("PostgreSQL ou MySQL") e **define** o MongoDB como o lado documental. Mas cabe justificar por que o modelo de documentos é o contraponto certo para *este* domínio — porque é disso que a comparação depende:

- **Chave-valor** (Redis) guarda e devolve, não agrega. Não responderia nenhuma das 16 perguntas.
- **Família de colunas** (Cassandra) é forte em escrita massiva e série temporal, não em ler um pedido inteiro nem em cruzar entidades.
- **Grafo** (Neo4j) resolveria bem a hierarquia de funcionários e a malha de territórios, mas essas são a periferia do domínio — o núcleo é transacional.

- **Documento** é o único que espelha o agregado que o Northwind tem de mais característico: **o pedido com seus itens**, lido inteiro, imutável depois de fechado.

E entre os documentais, o MongoDB tem *aggregation pipeline* com expressividade comparável à do SQL — sem isso, comparar as 16 perguntas nas duas tecnologias não seria uma comparação, seria uma demonstração de que uma delas não dá conta.

2. O domínio

Northwind Traders, uma distribuidora de alimentos importados. É um dataset clássico de ensino, e foi escolhido por três razões concretas, não por tradição:

1. **Tem processo de negócio completo:** catálogo, fornecedores, clientes, pedidos, itens, vendedores, territórios e entrega. Dá para fazer pergunta de negócio de verdade, não exercício sintético.
2. **Tem os dois casos que interessam à comparação:** um agregado natural (o pedido com seus itens, que se lê inteiro) e perguntas analíticas que cruzam entidades de formas variadas. Um dataset que só tivesse um dos dois tornaria a comparação enviesada por construção.
3. **É pequeno o suficiente para ser reproduzível** por qualquer avaliador com Docker, e grande o suficiente para ter irregularidades reais — que são o material da análise.

Volume: 14 tabelas, 3362 linhas, pedidos de 04/07/1996 a 06/05/1998.

3. A pergunta central

Qual a ferramenta certa para o trabalho?

Não "qual banco é melhor" — a pergunta assim posta não tem resposta. A formulação útil é: **qual é a forma dominante de acesso ao dado, e qual tecnologia serve melhor a ela?**

O trabalho responde isso com medição, não com opinião: as mesmas 16 perguntas de negócio são respondidas nas duas tecnologias, e a comparação sai da diferença.

4. Escopo

Entra:

- perfilamento e análise exploratória da base, com evidência executada
- modelo conceitual (ER), modelo relacional próprio com normalização justificada, constraints, índices e views
- modelo de documentos para MongoDB e migração idempotente
- 16 consultas SQL e 16 pipelines de agregação equivalentes
- benchmark comparativo e análise de sintaxe

Não entra, e é declarado:

- **margem de lucro** — a base não tem custo de aquisição de produto, só preço de venda. Toda análise financeira é de **receita**, nunca de margem.
- **cancelamento, devolução, pagamento e inadimplência** — não existem na base.
- **segmentação de clientes** — as tabelas existem e estão vazias (0 linhas).
- **escalabilidade** — 830 pedidos rodam em milissegundos nos dois bancos. O benchmark mede comportamento nesta escala, não capacidade de crescer.

5. Como o trabalho está organizado

Entrega	Conteúdo	Data
01	ER conceitual, Plano Híbrido, modelo relacional implementado	06/09/2026
02	Modelagem NoSQL e migração	13/09/2026
03	16 consultas SQL e 16 pipelines equivalentes	27/09/2026
04	Benchmark, relatório final e apresentação	08/10/2026

Os documentos seguem a mesma numeração. Esta entrega traz docs/00 a docs/04.

6. Duas regras que o trabalho seguiu

Nenhum número sem execução. Todo valor que aparece em qualquer documento foi produzido por um comando que está registrado ao lado dele, e a saída bruta está em apresentacao/evidencias/. Não há número estimado, arredondado de memória ou copiado de outro trabalho.

Reprodutibilidade é entregável. Um avaliador com Docker e este repositório levanta os dois bancos, carrega o Northwind, constrói o schema do projeto e regeira todas as evidências sem nenhum passo manual não documentado. O README.md tem os comandos na ordem.

01 — Análise de Negócio e Perfilamento do Northwind

Entrega 01 · Projeto Integrador Área 03 — Banco de Dados (IPOG) Autor: Giovanne Espíndola · Data: 26/08/2026 Todo número deste documento foi **executado** contra o banco. As saídas brutas estão em `apresentacao/evidencias/01-perfil-*.txt` e os scripts que as geraram estão versionados em `sql/01_exploracao.sql` e no notebook `etl/perfilamento.ipynb`.

1. O que foi feito, e como reproduzir

```
# 1. baixar o dump original
curl -fsSL https://raw.githubusercontent.com/pthom/northwind_psql/master/northwind.sql \
  -o sql/00_northwind_original.sql

# 2. carregar no schema public (a FONTE, que fica intocada)
docker compose exec -T postgres psql -U pi -d northwind \
  -v ON_ERROR_STOP=1 -f /sql/00_northwind_original.sql

# 3. perfilamento estrutural (6 arquivos de evidência)
uv run jupyter nbconvert --to notebook --execute --inplace etl/perfilamento.ipynb

# 4. perfilamento de negócio (1 arquivo de evidência)
docker compose exec -T postgres psql -U pi -d northwind \
  -v ON_ERROR_STOP=1 -f /sql/01_exploracao.sql
```

A carga terminou com **exit code 0** e nenhum erro: 14 **CREATE TABLE**, 3362 **INSERT**, 27 **ALTER TABLE** de constraint. O dump escolhido não contém **OWNER TO** nem **CREATE ROLE**, então carrega limpo em qualquer usuário — foi por isso que ele foi preferido a outras versões do Northwind que circulam.

Conferência cruzada: o log da carga reporta 3362 linhas inseridas; a soma de `count(*)` das 14 tabelas depois da carga dá exatamente **3362**. Nada se perdeu no caminho.

2. O processo de negócio, em prosa

A Northwind Traders é uma **importadora e distribuidora de alimentos especializados**. Ela não fabrica nada: compra de fornecedores no mundo inteiro e revende para clientes corporativos — restaurantes, mercados, delicatessens.

O ciclo que a base registra é o clássico **order-to-cash**, e ele acontece assim:

Um **fornecedor** (`suppliers`, 29 deles) vende à Northwind um conjunto de **produtos** (`products`, 77). Cada produto pertence a exatamente uma **categoria** (`categories`, 8 — bebidas, laticínios, frutos do mar...) e a exatamente um fornecedor. O produto tem preço de tabela, saldo em estoque, quantidade já encomendada ao fornecedor, um ponto de reposição e uma marca de descontinuado.

Do outro lado está o **cliente** (`customers`, 91), uma empresa identificada por um código de cinco letras — **ALFKI**, **BERGS**, **FISSA**. Quando ele compra, nasce um **pedido** (`orders`, 830). O pedido não guarda o que foi comprado: guarda o cabeçalho da venda — quem comprou, qual **funcionário** (`employees`, 9) fechou a venda, quando foi feito, para quando foi prometido, quando de fato saiu, por qual **transportadora** (`shippers`, 6) foi, quanto custou o frete e para onde foi entregue.

O que foi comprado está em `order_details` (2155 linhas): uma linha por produto dentro do pedido. Cada linha guarda quantidade, preço unitário **daquela venda** e desconto aplicado. É aqui que mora o dinheiro — a receita da empresa é $\text{quantidade} \times \text{preço unitário} \times (1 - \text{desconto})$, somada sobre estas 2155 linhas.

Existe ainda uma estrutura comercial de território: o país é dividido em **regiões** (`region`, 4) que se subdividem em **territórios** (`territories`, 53), e cada funcionário responde por um conjunto deles (`employee_territories`, 49 vínculos). E os funcionários se organizam em hierarquia: a coluna `reports_to` aponta para outro funcionário da mesma tabela.

Em uma frase: **a Northwind compra de fornecedores, cataloga em produtos, vende através de funcionários para clientes empresariais, e entrega via transportadoras.**

3. A base em números

3.1 Inventário

Evidência: `apresentacao/evidencias/01-perfil-01-inventario.txt`

tabela	linhas	colunas	papel
<code>order_details</code>	2 155	5	itens vendidos — onde está o dinheiro
<code>orders</code>	830	14	cabeçalho da venda
<code>territories</code>	53	3	subdivisão comercial
<code>us_states</code>	51	4	tabela de apoio sem nenhuma FK apontando para ela
<code>employee_territories</code>	49	2	associativa funcionário <-> território
<code>customers</code>	91	11	clientes empresariais
<code>products</code>	77	10	catálogo
<code>suppliers</code>	29	12	fornecedores
<code>employees</code>	9	18	equipe de vendas
<code>categories</code>	8	4	classificação de produto
<code>shippers</code>	6	3	transportadoras
<code>region</code>	4	2	região comercial
<code>customer_demographics</code>	0	2	vazia
<code>customer_customer_demo</code>	0	2	vazia
total	3 362	92	

A base é **pequena** — 3362 linhas, poucas centenas de kilobytes. Isso tem uma consequência que este trabalho precisa assumir desde já e está registrada na seção de limitações do benchmark: **não dá para medir escalabilidade aqui**. O que se pode medir é sobrecarga por consulta e diferença de modelo, não comportamento sob volume.

3.2 Período e volume

Evidência: seções 1 e 2 de `01-perfil-07-exploracao-negocio.txt`

Primeiro pedido	1996-07-04
Último pedido	1998-05-06

Janela de operação	671 dias (~22 meses)
Pedidos	830
Clientes que compraram	89 de 91 cadastrados
Vendedores ativos	9 de 9

ano	pedidos	clientes distintos	receita
1996	152	67	208 083,97
1997	408	86	617 085,20
1998	270	81	440 623,87

1996 e 1998 são **anos parciais** (a base começa em julho de 1996 e termina em maio de 1998). Comparar o total de 1996 com o de 1997 como se fosse queda ou crescimento seria erro de leitura — é a janela que é diferente. Só 1997 é ano cheio.

3.3 Dinheiro

Evidência: seções 3, 4, 5, 6 e 7 de `01-perfil-07-exploracao-negocio.txt`

Receita total do período	1 265 793,04
Ticket médio	1 525,05
Mediana do pedido	943,25
Menor pedido	12,50
Percentil 95	4 706,62
Maior pedido	16 387,50
Itens por pedido (mín / médio / máx)	1 / 2,60 / 25

A média (1 525,05) é **62% maior que a mediana** (943,25). Isso é a assinatura de uma distribuição com cauda longa à direita: a maioria dos pedidos é pequena e poucos pedidos muito grandes puxam a média para cima. Em uma apresentação, citar a média sozinha superestima o pedido típico — a mediana descreve melhor.

Receita por categoria — as 8 categorias, com participação:

categoria	produtos	receita	%
Beverages	12	267 868,18	21,2%
Dairy Products	10	234 507,29	18,5%
Confections	13	167 357,23	13,2%
Meat/Poultry	6	163 022,36	12,9%
Seafood	12	131 261,74	10,4%
Condiments	12	106 047,09	8,4%
Produce	5	99 984,58	7,9%
Grains/Cereals	7	95 744,59	7,6%

Duas categorias (Beverages + Dairy) fazem **39,7%** da receita. Meat/Poultry chama atenção pelo inverso: só **6 produtos** geram 12,9% — é a categoria de maior receita por item de catálogo.

Receita por país do cliente (top 5): USA 245 584,61 · Germany 230 284,63 · Austria 128 003,84 · Brazil 106 925,78 · France 81 358,32. Os dois primeiros somam **37,6%** — a Northwind é uma operação concentrada em

dois mercados.

4. Fraquezas de modelagem encontradas

Esta seção é o coração da justificativa para o schema `nw`. Cada item traz a evidência que o comprova.

4.1 Dinheiro em ponto flutuante — o erro mais grave

`order_details.unit_price`, `order_details.discount`, `orders.freight` e `products.unit_price` são do tipo `real` (ponto flutuante de 4 bytes).

Ponto flutuante binário **não representa exatamente** valores decimais como 0,05 ou 19,90 — pelo mesmo motivo que 1/3 não tem representação decimal finita. O erro é minúsculo em cada valor, mas soma-se ao longo da agregação.

A mesma soma, feita nos dois tipos (seção 3 da evidência):

receita_em_real		receita_em_numeric		diferenca
-----+-----+-----				
1265793.038653364		1265793.0395		-0.00084664

Menos de um centavo em 1,2 milhão. **O ponto não é o tamanho do erro, é a existência dele:** o valor em `real` nem sequer é estável — depende da ordem em que as linhas foram somadas. Num sistema financeiro real isso é inaceitável, e é o tipo de detalhe que separa quem copiou um dump de quem leu o schema.

Decisão para o `nw`: `numeric(10,2)` para valores monetários e `numeric(4,3)` para desconto.

4.2 Nenhuma regra de negócio no banco

Evidência: `01-perfil-03-chaves-e-constraints.txt`

tipo de constraint	quantidade
PRIMARY KEY	14
FOREIGN KEY	13
UNIQUE	0
CHECK	0
índices além dos de PK	0

A base tem integridade **referencial** (as FKs existem e, verificado por anti-join, **nenhuma das 13 tem um único registro órfão**), mas **nenhuma integridade de domínio**. Hoje o banco aceitaria, sem reclamar:

- quantidade vendida **negativa**
- desconto de **300%**
- preço unitário **negativo**
- `discontinued = 7`
- um pedido **enviado antes de ter sido feito**

Os dados atuais não violam nada disso — conferido na seção 17 da evidência, todas as sete faixas verificadas têm **0 violações**, e não há nenhuma data fora de ordem. Mas isso é sorte da carga, não garantia do modelo. **A constraint não existe para descrever o que os dados são; existe para impedir o que eles não podem virar.**

Faixas reais medidas, que fundamentam os CHECK do `nw`:

coluna	mínimo	máximo	CHECK proposto
<code>order_details.unit_price</code>	2,00	263,50	<code>>= 0</code> — zero registra brinde e amostra
<code>order_details.quantity</code>	1	130	<code>> 0</code>
<code>order_details.discount</code>	0,00	0,25	<code>>= 0 AND < 1</code>
<code>orders.freight</code>	0,02	1 007,64	<code>>= 0</code>
<code>products.units_in_stock</code>	0	125	<code>>= 0</code>
<code>products.discontinued</code>	0	1	vira boolean

4.3 Ausência de índice em toda coluna de junção

Os únicos 14 índices da base são os criados automaticamente pelas PKs. Nenhuma FK tem índice. Consequência prática: `order_details.order_id` — a coluna mais usada do banco, presente em toda consulta de faturamento — **não tem índice**, e todo **JOIN** sobre ela varre a tabela inteira.

Cardinalidade medida das principais colunas de junção (`01-perfil-05-cardinalidade-juncoes.txt`):

junção	linhas	distintos	média filhos/pai	seletividade
<code>order_details.order_id -> orders</code>	2 155	830	2,6	alta — bom candidato a índice
<code>order_details.product_id -> products</code>	2 155	77	28,0	média
<code>orders.customer_id -> customers</code>	830	89	9,3	alta
<code>orders.employee_id -> employees</code>	830	9	92,2	baixa — índice pode não ser usado
<code>orders.ship_via -> shippers</code>	830	3	276,7	muito baixa
<code>products.category_id -> categories</code>	77	8	9,6	baixa

Isso não significa "crie índice em tudo". `orders.ship_via` tem **3 valores distintos em 830 linhas** — um índice ali provavelmente seria ignorado pelo planejador, porque ler o índice e depois buscar 276 linhas na tabela é mais caro que varrer as 830 direto. Essa distinção é exatamente o que o **nw** precisa justificar índice a índice.

4.4 Redundância intencional vs. redundância acidental

Esta é a distinção que mais rende em banca, e a base tem um exemplo perfeito de cada lado.

Intencional — e correta. `order_details.unit_price` repete um dado que já existe em `products.unit_price`. Parece violação de normalização. Não é: é um **snapshot histórico**. O preço de catálogo muda; o preço pelo qual a venda aconteceu não pode mudar nunca — senão o faturamento do ano passado se altera sozinho quando alguém reajusta a tabela de preços.

A prova está nos dados (seção 12 da evidência): **662 dos 2155 itens (30,7%) têm preço de venda diferente do preço de catálogo atual**. Exemplo real:

produto	preço no catálogo hoje	preço cobrado na venda	data da venda
Alice Mutton	39,00	31,20	1996-07-25

Se `order_details` não guardasse o próprio preço, todo o histórico de receita seria recalculado a 39,00 e estaria errado.

Acidental — e discutível. `orders` guarda o endereço de entrega em seis colunas soltas: `ship_name`, `ship_address`, `ship_city`, `ship_region`, `ship_postal_code`, `ship_country`. Medido (seção 14):

- em **796 dos 830** pedidos (95,9%) o `ship_name` é idêntico ao `company_name` do cliente
- em **782 dos 830** (94,2%) `ship_address`, `ship_city` e `ship_country` são cópia literal do cadastro. Comparando as **seis** colunas `ship_*` — somando `ship_name`, `ship_region` e `ship_postal_code` — a coincidência cai para **748 (90,1%)**

Aqui a justificativa de snapshot é **fraca**: endereço de entrega não é um valor que precise ser congelado por razão contábil, e 90% do conteúdo é duplicação pura. Ainda assim, **48 pedidos (5,8%)** têm endereço de entrega diferente do cadastro pelo critério das três colunas, e **82 (9,9%)** pelo critério das seis — ou seja, a coluna não é inútil, ela cobre o caso de entrega em endereço alternativo.

A resposta madura não é "normalizar" nem "deixar como está": é reconhecer que faltou uma entidade. O modelo correto teria endereços do cliente como entidade própria, com o pedido referenciando qual endereço foi usado.

4.5 Um N:N que os dados desmentem

`employee_territories` tem PK composta (`employee_id`, `territory_id`) — a estrutura clássica de um relacionamento muitos-para-muitos. Os dados dizem outra coisa (seção 9):

vinculos	funcionarios_com_territorio	territorios_vinculados	max_funcionarios_por_territorio
49	9	49	1

São 49 vínculos para 49 territórios distintos, e **nenhum território tem mais de um funcionário**. Na prática o relacionamento é **1:N** (um funcionário atende vários territórios, cada território tem um único responsável), não N:N.

Isso não é necessariamente erro de modelagem — pode ser uma escolha deliberada de permitir N:N para o futuro. Mas é uma pergunta que a banca pode fazer, e a resposta precisa vir com o número na mão. Além disso, **4 dos 53 territórios não têm nenhum funcionário** — buraco de cobertura comercial.

4.6 Colunas de chave estrangeira que aceitam nulo

`orders.customer_id`, `orders.employee_id`, `orders.ship_via` e `orders.order_date` são todas **nullable**. Ou seja, o schema atual permite um pedido **sem cliente, sem vendedor, sem transportadora e sem data**. Nos dados carregados nenhuma dessas colunas tem nulo — mas, de novo, isso é o estado atual, não uma garantia.

`orders.shipped_date` nulo, por outro lado, é **legítimo e significativo**: os 21 pedidos com `shipped_date IS NULL` são pedidos que nunca foram enviados. Aqui o nulo carrega informação de negócio e deve continuar permitido.

4.7 Tipos frouxos e outros achados

achado	evidência	impacto
<code>order_id</code> é <code>smallint</code> (teto 32 767)	maior id atual: 11 077 , folga de 21 690	limite de crescimento embutido no tipo
<code>discontinued</code> é <code>integer</code> usado como booleano	valores observados: só 0 e 1	<code>boolean</code> expressa a intenção
<code>customer_id</code> é <code>varchar(5)</code> — chave natural	91 valores, todos com 5 letras	acopla o identificador ao nome da empresa

achado	evidência	impacto
<code>products.quantity_per_unit</code> é texto livre	ex.: "10 boxes x 20 bags"	não é consultável nem calculável
<code>us_states</code> (51 linhas) sem nenhuma FK	01-perfil-03	tabela ilha, não conectada ao modelo
<code>customer_demographics</code> e <code>customer_customer_demo</code> vazias	0 linhas	estrutura sem dado; N:N morto
<code>region</code> com nome genérico	4 linhas	colide com a coluna <code>region</code> , que existe em quatro tabelas

4.8 Nulos concentrados em colunas de endereço

Evidência: 01-perfil-04-nulos-por-coluna.txt — de 92 colunas, 11 têm nulo.

coluna	% nulo	leitura
<code>suppliers.homepage</code>	82,8%	base de 1996: quase ninguém tinha site
<code>suppliers.region</code>	69,0%	"region" só faz sentido em países federativos
<code>customers.region</code>	65,9%	idem
<code>orders.ship_region</code>	61,1%	idem, herdado do cliente
<code>suppliers.fax</code>	55,2%	nem toda empresa tinha fax
<code>employees.region</code>	44,4%	4 dos 9 funcionários são do escritório de Londres
<code>customers.fax</code>	24,2%	
<code>orders.shipped_date</code>	2,5%	21 pedidos nunca enviados — informação real
<code>orders.ship_postal_code</code>	2,3%	
<code>customers.postal_code</code>	1,1%	
<code>employees.reports_to</code>	11,1%	1 nulo: o topo da hierarquia

Dois desses nulos **não são falta de dado, são dado**: `reports_to` nulo identifica o presidente, e `shipped_date` nulo identifica pedido não enviado. Os demais são o que se chama de *nulo por inaplicabilidade* — "região" não se aplica a um endereço na Alemanha.

5. O que a base tem de bom

Um relatório que só aponta defeito é tão desequilibrado quanto um que só elogia. O Northwind acerta em coisas importantes:

- **Integridade referencial impecável.** As 13 FKs foram testadas por anti-join e **nenhuma tem um único órfão**. Nenhum item de pedido aponta para pedido inexistente, nenhum produto aponta para categoria inexistente.
- **Nenhum pedido vazio.** Os 830 pedidos têm pelo menos um item.
- **Nenhum produto encalhado no cadastro.** Todos os 77 produtos foram vendidos ao menos uma vez; todos os 29 fornecedores têm produto vendido.
- **Coerência temporal perfeita.** Zero pedidos enviados antes da data do pedido, zero prazos anteriores ao pedido, zero pedidos sem data.
- **PK composta onde deve.** `order_details(order_id, product_id)` e `employee_territories(employee_id, territory_id)` estão corretas.

- **O auto-relacionamento funciona.** A hierarquia de `reports_to` é consistente: Andrew Fuller (VP de Vendas) no topo com 5 subordinados diretos, Steven Buchanan (Gerente) com 3 — **três níveis**, sem ciclo.

6. Perguntas de negócio que a base responde

Lista fechada, que orienta as Entregas 03 e 04. Cada pergunta será respondida **duas vezes** — uma em SQL (`sql/queries/QNN.sql`) e uma em pipeline de agregação do MongoDB (`mongo/pipelines/PNN.js`) — e é essa duplicação que produz a análise comparativa e a base do benchmark.

#	Pergunta de negócio	Recurso técnico exercitado
01	Qual o faturamento por categoria de produto?	JOIN + agregação
02	Qual o ticket médio mês a mês?	agregação temporal
03	Que produtos nunca foram vendidos?	anti-join (<code>LEFT JOIN ... IS NULL</code>)
04	Quanto cada vendedor faturou?	JOIN + <code>GROUP BY</code>
05	Que produtos estão com estoque abaixo do ponto de reposição?	filtro + comparação entre colunas
06	Que clientes não compram há mais de 6 meses?	data + anti-join
07	Quais os 5 produtos mais vendidos dentro de cada categoria?	window function (<code>RANK</code>)
08	Como evolui o faturamento acumulado mês a mês?	running total (<code>SUM OVER</code>)
09	Qual a variação percentual de receita de um mês para o outro?	<code>LAG</code>
10	Quais clientes formam a curva ABC (20% que fazem 80%)?	<code>NTILE</code> / percentil
11	Como segmentar clientes por Recência, Frequência e Valor (RFM)?	CTEs encadeadas
12	Que produtos são comprados juntos no mesmo pedido?	self-join / <code>\$unwind</code> duplo
13	Qual o prazo médio de entrega por transportadora?	diferença de datas
14	Qual o impacto do desconto sobre a receita?	cálculo derivado
15	Quanto vende a equipe de cada gerente, somando os subordinados?	CTE recursiva vs <code>\$graphLookup</code>
16	Existe sazonalidade por trimestre?	<code>\$facet</code> / <code>GROUPING SETS</code>

As perguntas 07 a 16 são as que demonstram domínio além de `GROUP BY` — e são justamente as que expõem as diferenças reais entre os dois modelos de banco.

Perguntas que a base NÃO responde, e que é melhor declarar do que descobrir na frente da banca: não há custo de aquisição do produto (logo, **não se calcula margem, só receita**), não há cancelamento nem devolução, não há pagamento nem inadimplência, e a segmentação de cliente (`customer_demographics`) está vazia.

7. Limitações declaradas

1. **Volume.** 830 pedidos e 2155 itens. Qualquer consulta roda em milissegundos nos dois bancos. O benchmark da Entrega 04 mede **sobrecarga por consulta e diferença de modelo**, não escalabilidade.
2. **Recorte temporal.** 1996 e 1998 são anos parciais; só 1997 é ano cheio.
3. **Sem margem.** Sem custo de produto, toda análise de rentabilidade é sobre receita, não lucro.
4. **Duas tabelas vazias.** `customer_demographics` e `customer_customer_demo` não sustentam nenhuma análise.
5. **Dados sintéticos.** O Northwind é uma base de demonstração da Microsoft, não um extrato de operação real. Padrões encontrados descrevem o dataset, não o mercado de alimentos dos anos 90.

8. Próximo passo

Com a base perfilada, a Entrega 01 segue para: diagrama ER conceitual (**docs/02-***), Plano Híbrido (**docs/03-***) e o schema **nw** (**docs/04-***) — que enfrentará, um a um e com justificativa, os oito grupos de fraqueza da seção 4.

02 — Modelo Conceitual (ER) do Northwind

Entrega 01 · Projeto Integrador Área 03 — Banco de Dados (IPOG) Autor: Giovanne Espíndola · Data: 30/08/2026 Diagrama: docs/diagramas/er-conceitual.drawio · exportação: docs/diagramas/er-conceitual.png Cada cardinalidade abaixo vem acompanhada do número que a sustenta, medido em apresentacao/evidencias/01-perfil-05-cardinalidade-juncoes.txt e reconferido por etl/valida_conceitual.py.

Modelo Entidade-Relacionamento CONCEITUAL — Northwind Traders

Distribuidora de alimentos · entidades, relacionamentos e cardinalidades.

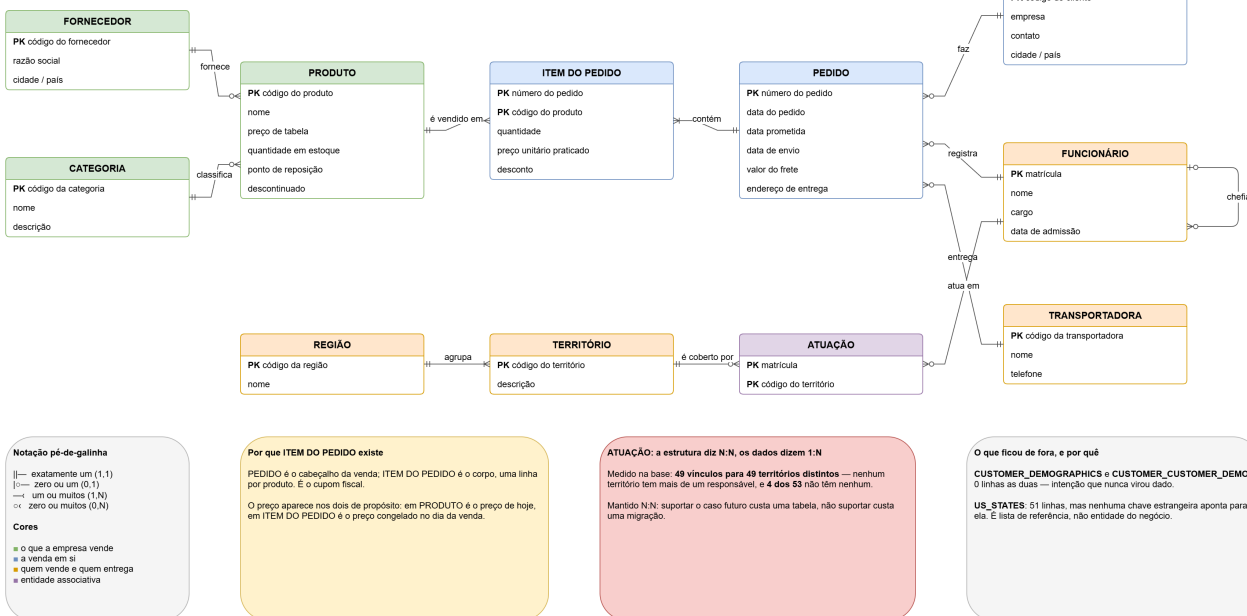


Figura 1 — Modelo Entidade-Relacionamento conceitual do Northwind Traders.

1. O arquivo do diagrama

O diagrama está em formato **.drawio** — XML nativo do draw.io (diagrams.net). Duas consequências práticas: ele abre e continua editável no aplicativo, e é texto, então versiona no Git como qualquer outro arquivo de código (um **git diff** mostra que uma cardinalidade mudou).

Para abrir: <https://app.diagrams.net> -> *Open Existing Diagram*, ou a extensão **Draw.io Integration** no VS Code, que abre o arquivo direto no editor.

O diagrama não é só desenho: **etl/valida_conceitual.py** confere cada afirmação dele contra o banco — as 11 entidades e seus atributos existem no schema **public**, os identificadores batem com as chaves primárias reais, as cardinalidades conferem com as junções medidas, e as 14 tabelas da base estão todas classificadas (11 modeladas, 3 excluídas com motivo). O script sai com código 1 se qualquer verificação falhar, e a saída fica em **apresentacao/evidencias/02-validacao-er-conceitual.txt**.

```
uv run python etl/valida_conceitual.py
```

2. As onze entidades, em três blocos

O modelo é mais fácil de defender se apresentado em blocos, não como uma lista de onze caixas.

Bloco 1 — o que a empresa vende. FORNECEDOR vende PRODUTO, e cada produto pertence a uma CATEGORIA. É o catálogo.

Bloco 2 — a venda. CLIENTE faz PEDIDO, e o pedido é composto de ITEM DO PEDIDO, um por produto vendido. É a transação.

Bloco 3 — quem vende e quem entrega. FUNCIONÁRIO registra o pedido, TRANSPORTADORA entrega. Funcionários atuam em TERRITÓRIO (pela entidade associativa ATUAÇÃO), e territórios se agrupam em REGIÃO. É a operação.

O modelo tem entidades, relacionamentos, cardinalidades e os atributos que identificam ou descrevem cada entidade — e nada além disso. O que ele deliberadamente não mostra está na §5.

3. Justificativa de cada cardinalidade

São **10 relacionamentos**, e a tabela abaixo tem 10 linhas. No desenho eles aparecem como **11 linhas**, porque o N:N entre FUNCIONÁRIO e TERRITÓRIO passa pela entidade associativa ATUAÇÃO e por isso é traçado em duas pernas. No modelo relacional viram **11 chaves estrangeiras**, pelo mesmo motivo.

A notação é pé-de-galinha. **(1,1)** é "exatamente um", **(0,1)** "zero ou um", **(1,N)** "um ou muitos", **(0,N)** "zero ou muitos".

Relacionamento	Cardinalidade	Regra de negócio que a justifica	Evidência medida
CLIENTE faz PEDIDO	(1,1) — (0,N)	Todo pedido é de alguém; um cliente cadastrado pode ainda não ter comprado	830 pedidos, nenhum sem cliente · 89 dos 91 clientes têm pedido: 2 nunca compraram
FUNCIONÁRIO registra PEDIDO	(1,1) — (0,N)	Toda venda tem um responsável; um recém-contratado ainda não vendeu	9 de 9 funcionários com pedido · média de 92,2 pedidos por funcionário
TRANSPORTADORA entrega PEDIDO	(1,1) — (0,N)	O pedido sai por uma transportadora; nem toda transportadora cadastrada é usada	apenas 3 das 6 transportadoras aparecem (50%)
PEDIDO contém ITEM DO PEDIDO	(1,1) — (1,N)	Pedido sem nenhum item não é uma venda — é um formulário em branco	2155 itens em 830 pedidos, média 2,6 · os 830 pedidos têm item, nenhum vazio
PRODUTO é vendido em ITEM DO PEDIDO	(1,1) — (0,N)	Um produto do catálogo pode nunca ter sido vendido	77 de 77 produtos vendidos, média 28 itens por produto
CATEGORIA classifica PRODUTO	(1,1) — (0,N)	Todo produto tem categoria; categoria nova pode estar vazia	8 categorias, média 9,6 produtos
FORNECEDOR fornece PRODUTO	(1,1) — (0,N)	Todo produto tem origem; fornecedor pode estar sem produto ativo	29 fornecedores, média 2,7 produtos
REGIÃO agrupa TERRITÓRIO	(1,1) — (1,N)	Região comercial sem nenhum território não existe como região	4 regiões, 53 territórios, média 13,2 , todas com território
FUNCIONÁRIO atua em TERRITÓRIO	(0,N) — (0,N)	Um funcionário cobre vários territórios; um território pode ter mais de um responsável	49 vínculos · 9 funcionários, média 5,4 territórios cada · 4 dos 53 territórios sem ninguém
FUNCIONÁRIO chefia FUNCIONÁRIO	(0,1) — (0,N)	Cada um responde a no máximo um superior; o topo não responde a ninguém	9 funcionários, 1 sem chefe (o topo) e 2 chefes distintos

O padrão **(1,1) — (0,N)** domina a tabela, e a leitura é sempre a mesma: **o lado filho é obrigatório, o lado pai é opcional**. Um pedido tem que ter cliente; um cliente não tem que ter pedido.

Um detalhe que a banca pode cobrar: cinco dessas colunas de junção (`orders.customer_id`, `orders.employee_id`, `orders.ship_via`, `products.supplier_id`, `products.category_id`) são **nuláveis**

no **dump original**, embora não tenham nenhum nulo nos dados. O modelo conceitual as trata como obrigatórias porque a regra de negócio é obrigatória — o dump é permissivo, o negócio não é. É exatamente esse aperto que o schema **nw** vai transformar em **NOT NULL**.

4. As três decisões do modelo, e o que foi descartado

4.1 ITEM DO PEDIDO é entidade, não atributo de PEDIDO

Descartado: guardar os produtos vendidos dentro do próprio pedido, em campos repetidos (**produto_1**, **produto_2**, ...) ou numa lista.

Por quê: um pedido tem número variável de produtos — na base, de 1 a dezenas. Campo repetido obriga a fixar um teto arbitrário e viola a 1FN. Separar em entidade própria é o que permite perguntar "quanto vendemos deste produto" sem varrer coluna por coluna.

A identificação de **ITEM DO PEDIDO** é **composta**: o par (pedido, produto). Não existe item fora de um pedido, e o mesmo produto não aparece duas vezes no mesmo pedido — a chave composta impõe as duas regras de uma vez.

4.2 ATUAÇÃO é N:N na estrutura, mesmo os dados dizendo 1:N

Os dados medidos: **49 vínculos para 49 territórios distintos**, nenhum território com mais de um responsável. Na prática, hoje, o relacionamento é 1:N.

Descartado: simplificar para 1:N, colocando o responsável dentro de **TERRITÓRIO**.

Por quê: a assimetria de custo. Suportar o N:N custa uma entidade associativa. Não suportar custa uma migração de schema no dia em que dois vendedores dividirem uma praça — o que é uma decisão comercial trivial, não uma mudança de negócio. Modelar para o caso mais geral, quando o custo é uma tabela, é a escolha defensável.

Isto é material de banca: a pergunta "por que N:N se os dados são 1:N?" tem resposta pronta, com o número na mão.

4.3 Três tabelas ficaram de fora

customer_demographics e **customer_customer_demo** existem na base original e estão **vazias** — **0 linhas as duas**.

Descartado: incluí-las no modelo "porque estão no dump".

Por quê: entidade sem nenhuma instância não descreve o domínio, descreve uma intenção que nunca virou dado. Modelo conceitual é retrato do negócio que existe.

A terceira é **us_states**, e o motivo é outro: ela tem **51 linhas**, mas **nenhuma chave estrangeira aponta para ela** — as colunas **region** de **customers**, **employees** e **suppliers** são texto livre, não referência. É uma lista de apoio geográfico que veio junto no dump, não uma entidade do negócio de distribuição.

As três exclusões ficam **declaradas** no próprio diagrama, numa nota — o avaliador vê que foi decisão, não descuido.

5. O que este modelo deliberadamente não mostra

Tipo de dado, tamanho de campo, índice e constraint. Tudo isso é modelo lógico e físico, e entra em **docs/04** junto com o schema **nw**.

A separação não é formalismo: o modelo conceitual é o documento que se discute com quem entende do negócio e não entende de banco. No momento em que aparece **VARCHAR(40)** no diagrama, essa conversa acaba.

Dois coisas no diagrama parecem exceção e não são. Os identificadores aparecem — o **PK** nas caixas — porque **identificar é atributo conceitual**: dizer que ITEM DO PEDIDO se identifica pelo par (pedido, produto) é uma regra de negócio, o mesmo produto não entra duas vezes no mesmo pedido. E ATUAÇÃO aparece como caixa porque **entidade associativa é conceito de ER**, não artefato físico: é assim que a notação representa um N:N. A chave estrangeira, que é o mecanismo que implementa as duas coisas, essa sim só existe no modelo lógico.

6. Próximo passo

Plano Híbrido (**docs/03**) e schema **nw** (**docs/04**), onde cada cardinalidade desta tabela vira uma chave estrangeira e cada regra de negócio citada aqui vira uma constraint verificável.

03 — Plano Híbrido: PostgreSQL e MongoDB no mesmo projeto

Entrega 01 · Projeto Integrador Área 03 — Banco de Dados (IPOG) Autor: Giovanne Espíndola · Data: 31/08/2026 Diagrama: docs/diagramas/arquitetura-hibrida.drawio · exportação: docs/diagramas/arquitetura-hibrida.png

1. A pergunta que este documento responde

O projeto inteiro gira em torno de "**qual a ferramenta certa para o trabalho?**". Este plano define como os dois bancos vão conviver e, principalmente, **como a comparação entre eles vai ser feita de um jeito que o resultado signifique alguma coisa.**

Como docs/00 §1 já enquadra, a pergunta útil não é qual banco é melhor, e sim **qual é a forma dominante de acesso ao dado.** Este documento define o arranjo que permite medir isso.

2. O que cada banco faz bem — por critério, não por adjetivo

	PostgreSQL	MongoDB
Unidade natural	a linha, ligada por chave	o documento, lido inteiro
Integridade	declarativa: o banco recusa o que viola	da aplicação, ou por <i>validator</i> opcional
Pergunta imprevista	JOIN responde o que ninguém antecipou	precisa de \$lookup , ou de remodelar
Leitura do agregado	reconstruir exige junção	uma leitura, sem junção
Escala	vertical, réplicas de leitura e particionamento declarativo	horizontal, por <i>sharding</i>
Transação multi-entidade	nativa desde sempre	ACID multi-documento desde a 4.0, mas exige replica set

A linha que mais importa é a terceira. **Modelo relacional é o que permite fazer uma pergunta que não foi prevista na hora de modelar.** Modelo documental é o que permite servir rápido a pergunta que *foi* prevista.

A última linha merece precisão, porque é onde é fácil errar nos dois sentidos. O MongoDB **tem** transação ACID multi-documento desde a versão 4.0 — dizer que não tem está errado. Mas ela exige *replica set*, e o container deste projeto sobe em **nó único**, então aqui ela não está disponível: uma tentativa devolve **This MongoDB deployment does not support retryable writes**. Na prática do modelo documental isso pesa menos do que parece, porque **o agregado costuma ser a própria fronteira transacional** — a escrita de um documento é atômica, e o pedido com seus itens é um documento só.

Arquitetura híbrida — PostgreSQL como fonte, MongoDB como cópia documental

Fluxo de mão única. O relacional registra; o documental serve. As mesmas 16 perguntas rodam nos dois.

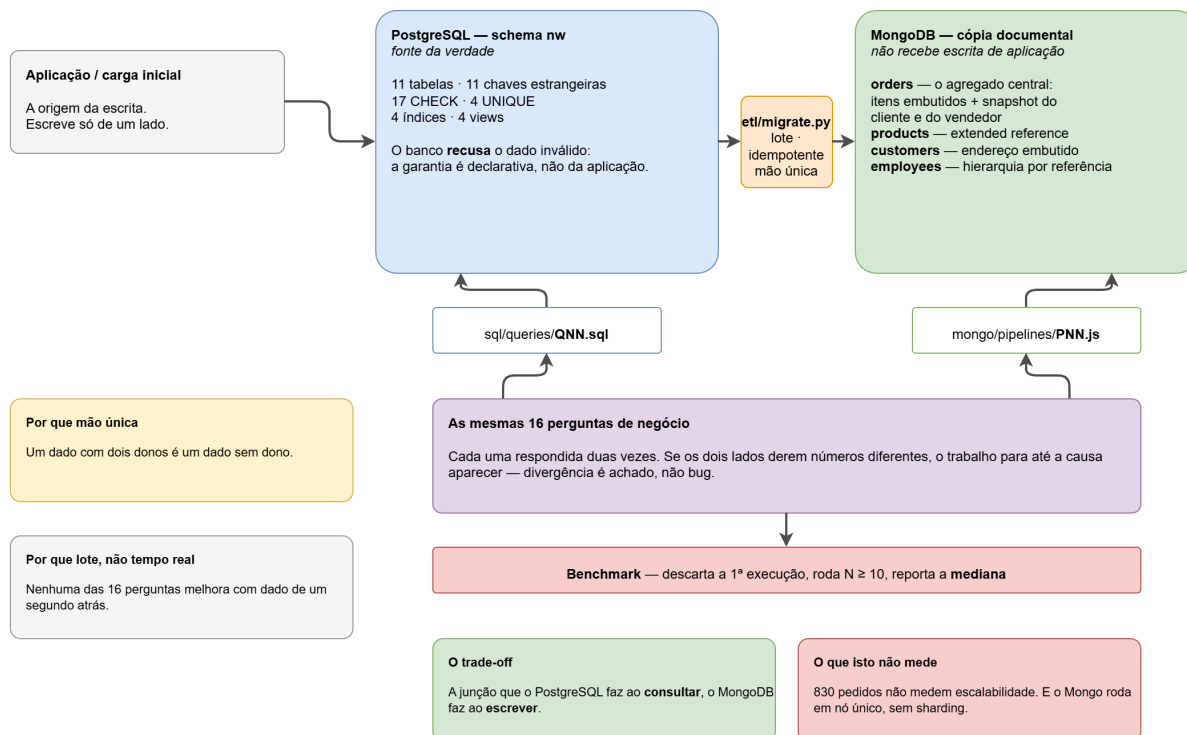


Figura 2 — Arquitetura híbrida: PostgreSQL como fonte, MongoDB como cópia documental.

3. A arquitetura escolhida

O PostgreSQL é a fonte da verdade. É onde a venda acontece, onde as **17 CHECK, 4 UNIQUE e 11 chaves estrangeiras** impedem o dado inválido de entrar, e onde a transação garante que pedido e itens nascem juntos ou não nascem.

O MongoDB será uma cópia orientada a documento, alimentada pelo `etl/migrate.py` que a Entrega 02 constrói. Ele não recebe escrita de aplicação neste projeto. Nesta entrega o container está no ar e vazio: o que existe aqui é o plano, e é ele que a Entrega 02 executa.

Por que o fluxo é de mão única

Sincronizar nos dois sentidos exigiria decidir quem ganha quando os dois lados mudam o mesmo dado — e qualquer resposta a isso é arbitrária. **Um dado com dois donos é um dado sem dono.** Mão única elimina a pergunta.

Por que ETL em lote, e não replicação em tempo real

Captura de mudança em tempo real (CDC) existe e funciona — Debezium lendo o WAL do PostgreSQL, Kafka no meio. Seria a escolha certa para um sistema com requisito de latência.

Este projeto não tem esse requisito, e a complexidade seria maior que o problema: mais dois serviços para o avaliador subir, e nenhuma das 16 perguntas de negócio melhora com dados de um segundo atrás. **Lote idempotente, que qualquer um roda com um comando, vale mais aqui do que arquitetura de streaming.**

4. O desenho do documento — o que muda de um lado para o outro

A decisão fecha na Entrega 02, mas o caminho já está definido, e ele é o que torna a comparação interessante:

Coleção	Estratégia	Por quê
<code>orders</code>	agregado central: itens embutidos como array, com <i>snapshot</i> do cliente e do vendedor	um pedido é lido inteiro ou não é lido; item não existe fora do pedido; nada muda depois que o pedido fecha
<code>products</code>	categoria e fornecedor como <i>extended reference</i>	quem lê um produto quase sempre quer o nome da categoria junto — mas não o resto dela
<code>customers</code>	endereço embutido	endereço não é consultado sozinho
<code>employees</code>	hierarquia por referência (<code>reports_to</code>)	árvore de profundidade variável; embutir criaria aninhamento sem fim

A tradução que importa: as 830 linhas de `orders` mais as 2155 de `order_items` viram **830 documentos**. A junção que o PostgreSQL faz em tempo de consulta, o MongoDB faz em tempo de escrita — uma vez só, no ETL.

É aí que está o *trade-off* que o experimento vai medir: o documental **paga antecipado** o custo da junção. A hipótese é que isso o favoreça na leitura prevista e o penalize no cruzamento imprevisto — mas isso é o que a Entrega 04 tem que mostrar, não algo a afirmar agora.

Uma decisão fica em aberto de propósito e será fechada na Entrega 02: se os itens embutidos em `orders` **carregarem também a categoria do produto**, as perguntas 01 e 07 (faturamento e top-5 por categoria) se resolvem sem `$lookup`; se não carregarem, precisarão dele. Os dois caminhos são defensáveis, e a escolha é justamente o tipo de coisa que a comparação existe para iluminar.

5. Como a comparação vai ser feita

Um conjunto único de perguntas, respondido duas vezes. As 16 perguntas de negócio de `docs/01` §6 **vão virar**, na Entrega 03, 16 consultas SQL (`sql/queries/QNN.sql`) e 16 pipelines de agregação (`mongo/pipelines/PNN.js`). Q07 e P07 responderão exatamente à mesma pergunta.

Isso produz três entregáveis de uma vez: as consultas da Entrega 03, a análise comparativa de sintaxe, e a base do benchmark da Entrega 04.

A regra que sustenta o resultado: se QNN e PNN derem números diferentes, **o trabalho para até a causa ser encontrada**. Divergência não é bug para esconder, é material de análise — normalmente ela revela uma diferença real de semântica entre os dois modelos, e essa diferença é o achado.

Protocolo do benchmark: descartar a primeira execução (que paga o aquecimento de cache), rodar $N \geq 10$, reportar a **mediana** — não a média, que um único pico distorce. Mesmo container, mesma máquina, sem carga concorrente.

6. O que este experimento NÃO prova — declarado antes de medir

Esta seção existe para que nenhuma conclusão seja mais forte do que os dados permitem.

1. **Volume.** São 830 pedidos e 2155 itens. Tudo roda em milissegundos nos dois bancos. O benchmark **não prova escalabilidade**; mede como cada tecnologia se comporta nesta escala e como cada consulta se

escreve.

2. **Nó único.** O MongoDB roda sem *sharding* e sem *replica set*. A principal vantagem estrutural do documental — distribuir por muitas máquinas — **não é exercitada**. Comparar assim provavelmente favorece o relacional, e é honesto dizer antes de medir. (A ressalva: com 830 pedidos, *sharding* não mudaria o resultado de todo modo.)
3. **Sem concorrência.** Mede-se latência de consulta isolada, não vazão sob carga. São coisas diferentes.
4. **Sem custo de produto.** A base não tem custo de aquisição, então nenhuma pergunta calcula margem — só receita.
5. **Sem transação multi-documento no MongoDB.** Ela existe no produto desde a 4.0, mas exige *replica set*, e o nó único não oferece. Então a comparação de garantias transacionais entre os dois lados não é feita em pé de igualdade — está declarado aqui para não virar surpresa.

7. O critério que vai decidir a resposta final

No fim, "qual a ferramenta certa" será respondido por esta matriz, preenchida com medição na Entrega 04:

Critério	Vence quem
Servir um pedido inteiro para uma tela ou API	menos junção
Responder pergunta analítica não prevista	mais expressividade de consulta
Impedir dado inválido de entrar	mais garantia declarativa
Clareza de quem escreve a consulta	menos linhas para o mesmo resultado
Custo de mudar o modelo depois	menos migração

A hipótese que o projeto vai testar — e que pode ser refutada pelos dados: para o Northwind, o pedido é um agregado de livro-texto e o MongoDB deve vencer na leitura operacional; mas as 16 perguntas cruzam categoria, tempo, vendedor e cliente de formas variadas, e aí o **JOIN** deve vencer. Se for isso, a resposta correta não é escolher um — é **usar cada um para o que ele é**, que é a razão de o plano se chamar híbrido.

8. Próximo passo

Entrega 02: implementar o modelo de documentos aqui esboçado, escrever os *validators*, construir `etl/migrate.py` idempotente e validar que os totais batem nos dois bancos.

04 — Modelo Relacional: o schema nw

*Entrega 01 · Projeto Integrador Área 03 — Banco de Dados (IPOG) Autor: Giovanne Espíndola · Data: 31/08/2026 Scripts: `sql/10_ddl.sql` · `sql/20_load.sql` · `sql/30_indexes.sql` · `sql/40_views.sql`
Diagrama: `docs/diagramas/er-logico.png` · fonte: `docs/diagramas/nw-schema.dbml` Evidências: `apresentacao/evidencias/03-validacao-carga-nw.txt` e `04-indices-e-views.txt`*

1. Por que existe um segundo schema

O schema `public` guarda o Northwind original, **intocado**. O schema `nw` é o modelo deste projeto.

A separação não é capricho: sem ela, "implementar um modelo relacional" viraria "carregar um dump pronto", e não haveria nenhuma decisão de modelagem para defender. Com ela, cada diferença entre os dois schemas é uma decisão registrada — e o original continua ali do lado para comparação.

O `nw` implementa as 11 entidades de `docs/02` e enfrenta, uma a uma, as fraquezas medidas em `docs/01` §4 — corrigindo a maioria e **declarando** as que decidiu não corrigir.

Conceitual (docs/02)	Tabela em nw	Origem em public
FORNECEDOR	<code>suppliers</code>	<code>suppliers</code>
CATEGORIA	<code>categories</code>	<code>categories</code>
PRODUTO	<code>products</code>	<code>products</code>
ITEM DO PEDIDO	<code>order_items</code>	<code>order_details</code>
PEDIDO	<code>orders</code>	<code>orders</code>
CLIENTE	<code>customers</code>	<code>customers</code>
FUNCIONÁRIO	<code>employees</code>	<code>employees</code>
TRANSPORTADORA	<code>shippers</code>	<code>shippers</code>
TERRITÓRIO	<code>territories</code>	<code>territories</code>
REGIÃO	<code>regions</code>	<code>region</code>
ATUAÇÃO	<code>employee_territories</code>	<code>employee_territories</code>

Duas tabelas mudaram de nome. `order_details` virou `order_items` porque "detalhe" não diz o que a linha é, e a entidade conceitual chama-se ITEM DO PEDIDO. `region` virou `regions` por consistência com o resto — e porque `region` no singular colide com a coluna `region` que existe em quatro tabelas.

Três colunas também mudaram de nome, sempre pelo mesmo motivo — o nome antigo descrevia o mecanismo, não o significado:

Em public	Em nw
<code>orders.ship_via</code>	<code>orders.shipper_id</code>
<code>region.region_description</code>	<code>regions.region_name</code>
<code>territories.territory_description</code>	<code>territories.territory_name</code>

O ER lógico

`docs/diagramas/er-logico.png` mostra o mesmo modelo com o que o conceitual deliberadamente omitia: **tipo de dado, tamanho, obrigatoriedade e chave estrangeira**. Colocados lado a lado, os dois diagramas

explicam sozinhos por que existem dois — o conceitual se discute com quem entende do negócio, o lógico com quem vai construir.

Ele não foi desenhado à mão: `docs/diagramas/nw-schema.dbml` é **gerado do banco** com `db2dbml`. Mas "gerado" não é garantia — basta o schema mudar depois para o diagrama entregue passar a mentir. Por isso a correspondência é **verificada**, e não presumida: `etl/valida_dbml.py` confere as 11 tabelas, as **81 colunas** com nome, ordem, tipo e **obrigatoriedade**, e as **11 chaves estrangeiras** — cada uma com origem e destino — contra o `information_schema`, saindo com código 1 em qualquer divergência.

O validador foi testado por mutação em cinco frentes: alterando um tipo, removendo uma coluna, removendo uma chave estrangeira, deixando uma FK com o nome certo apontando para a coluna errada, e marcando como obrigatória uma coluna que não é. Acusou as cinco. As duas últimas entraram depois de uma revisão flagrar que o validador não as via.

```
npx -p @dbml/cli db2dbml postgres \  
'postgresql://pi:pi@localhost:5432/northwind?schemas=nw' -o docs/diagramas/nw-schema.dbml  
uv run python etl/valida_dbml.py      # evidência: 05-validacao-er-logico.txt
```

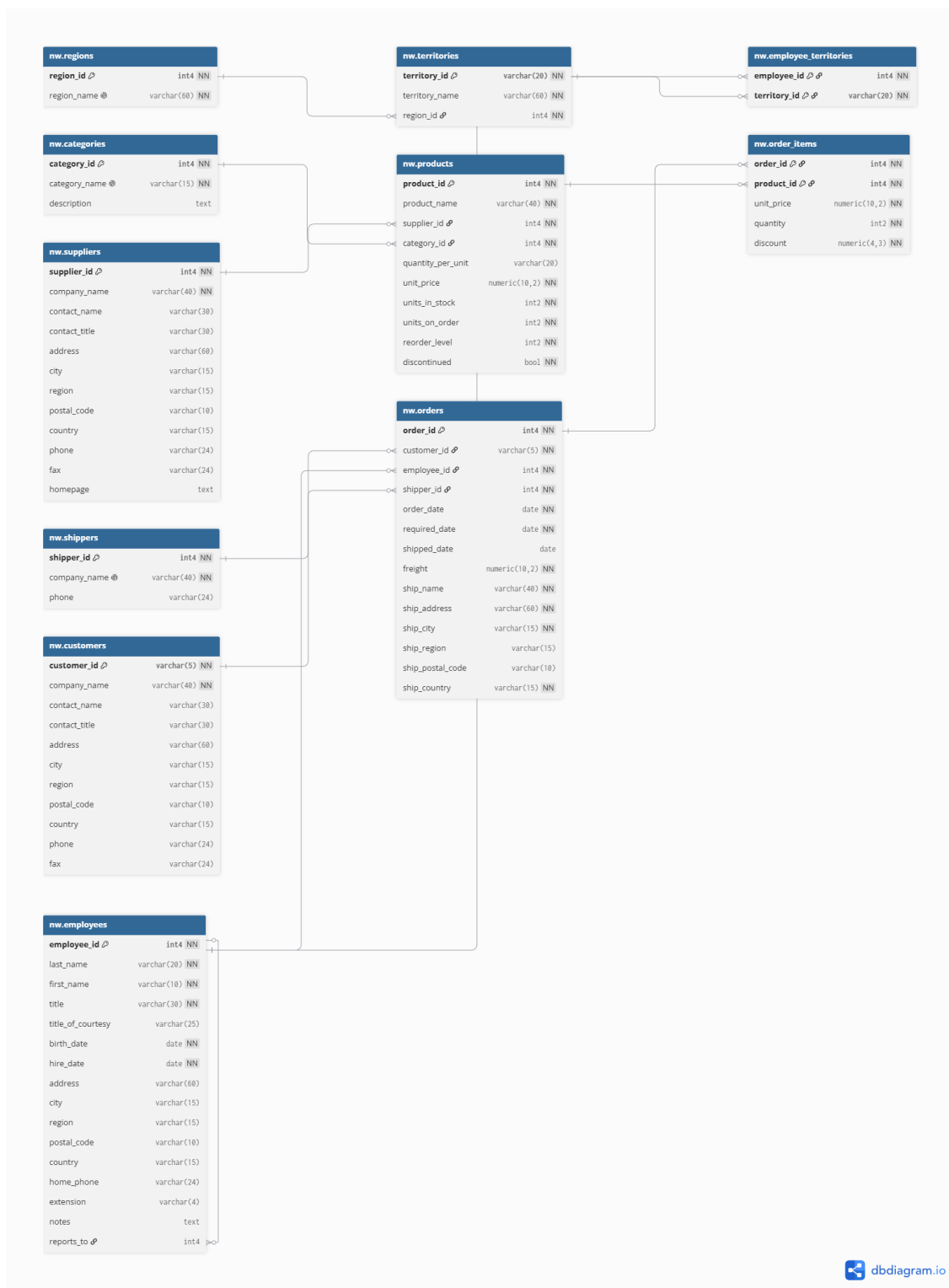


Figura 3 — Modelo Entidade-Relacionamento lógico do schema *nw*, gerado do banco.

2. O que o banco passou a impedir

Esta é a mudança mais importante, e cabe numa tabela:

	public (origem)	nw (projeto)
PRIMARY KEY	14	11
FOREIGN KEY	13	11

	public (origem)	nw (projeto)
UNIQUE	0	4
CHECK	0	17
Colunas obrigatórias que antes aceitavam nulo	—	19

As 19 obrigatórias saem de três grupos, e nenhuma é gosto pessoal: **as cinco colunas de junção** (`orders.customer_id`, `employee_id`, `shipper_id`, `products.supplier_id`, `category_id`) — pedido sem cliente ou produto sem categoria não existe no negócio; **as datas e os valores do pedido** (`order_date`, `required_date`, `freight` e o endereço de entrega) — pedido sem promessa de entrega não é pedido fechado numa distribuidora; e **os campos de identificação e preço** de produtos e funcionários.

Duas ficaram de fora **de propósito, depois de revisão**: `categories.description` e `products.quantity_per_unit` estavam obrigatórias só porque os dados de hoje as preenchem — que é exatamente o erro que este documento critica duas seções adiante. Categoria nova sem descrição e produto vendido a granel, que não tem embalagem para descrever, são cadastros legítimos. No caso de `quantity_per_unit` havia ainda uma contradição interna: a §3 declara a coluna uma violação de 1FN mantida "porque nenhuma das 16 perguntas usa esse campo", e o schema a exigia.

(Medido em `03-validacao-carga-nw.txt` §6 e §7. As PKs caem de 14 para 11 porque três tabelas ficaram fora — as duas vazias e a `us_states`. Já as FKs caem de 13 para 11 por causa de **uma só**: `customer_customer_demo` tinha duas; `us_states` e `customer_demographics` não tinham nenhuma.)

O banco de origem tinha integridade **referencial** perfeita e **nenhuma** integridade de domínio: aceitaria quantidade negativa, desconto de 300% e pedido enviado antes de ter sido feito. Agora não aceita.

O ponto que vale repetir na defesa: **a constraint não existe para descrever o que os dados são, existe para impedir o que eles não podem virar**. Os 3311 registros passaram por todas as 17 regras novas sem uma única violação — a carga rodou com `ON_ERROR_STOP=1` e saiu com código 0.

E o schema opera, não só carrega

As sete chaves sintéticas são **GENERATED BY DEFAULT AS IDENTITY**. Sem isso o modelo aceitaria a carga e depois não conseguiria registrar **uma venda nova**, porque ninguém saberia qual id usar — teria que calcular `max(id)+1` na mão, que além de trabalhoso não é seguro com dois usuários ao mesmo tempo.

BY DEFAULT e não **ALWAYS**: **BY DEFAULT** deixa a carga inserir os ids que vieram da origem e permite que uma inserção normal, sem id, receba o próximo número. Com **ALWAYS** toda instrução da carga precisaria de **OVERRIDING SYSTEM VALUE**.

Como a carga traz os ids prontos, as sequences ficariam paradas em 1 e o próximo **INSERT** colidiria. Por isso `20_load.sql` termina com `setval` reposicionando cada uma no maior id já usado — depois de uma carga limpa, a de `orders` fica em **11077**, que é o maior `order_id` da base. Testado numa transação revertida: um pedido inserido sem `order_id` recebeu **11078**, o número seguinte.

Três colunas ficaram para trás, de propósito

`categories.picture` e `employees.photo` são **bytea** de **comprimento zero** no dump — coluna declarada, conteúdo nenhum. `employees.photo_path` aponta para um caminho de rede de 1996 que não existe.

Nenhuma outra coluna se perdeu, e isso é **verificado, não afirmado**: a §8 de `21_validacao_carga.sql` compara as colunas das duas pontas contra essa lista de três e marca qualquer outra ausência como perda silenciosa. Foi assim que `employees.title_of_courtesy` (Mr., Mrs., Dr.) voltou para o modelo depois de ter sido esquecida na primeira versão.

Uma regra que precisou afrouxar

O **CHECK** de `customers.customer_id` começou como `^[A-Z]{5}$` — cinco letras maiúsculas, que é o formato dos 91 códigos da base. Está errado como regra: **ALFKI** vem de *Alfreds Futterkiste*, e quando duas empresas dão as mesmas cinco letras o desempate usual é trocar uma por dígito. O **CHECK** fecharia a porta para um cadastro legítimo. Ficou `^[A-Z0-9]{5}$`: prende o comprimento, que é regra real, e libera o desempate.

Uma regra que quase virou invenção

O primeiro rascunho tinha **UNIQUE** (`product_name`). Os 77 nomes do dump são distintos, então a constraint passava — mas ela **não é regra do negócio**: nada impede dois fornecedores de vender "Mozzarella". Foi trocada por **UNIQUE** (`supplier_id`, `product_name`), que é a regra defensável: *o mesmo fornecedor não cadastra o mesmo produto duas vezes*.

O caso oposto está a duas tabelas de distância e serve de contraprova: em **territories** **não** existe unique no nome, porque a base tem dois territórios chamados "New York" (10019 e 10038) — os dois atendidos pelo **mesmo** vendedor, Steven Buchanan. O nome se repete; o território, não. Mesmo critério, resultados opostos — é o dado que decide, não o hábito.

As correções de tipo

Coluna	Era	Virou	Por quê
Todos os valores em dinheiro	<code>real</code>	<code>numeric(10,2)</code>	ponto flutuante não representa 0,05 exatamente; o erro depende da ordem da soma
<code>order_items.discount</code>	<code>real</code>	<code>numeric(4,3)</code>	mesmo motivo, com a precisão que a regra de negócio pede
<code>products.discontinued</code>	<code>integer</code> 0/1	<code>boolean</code>	o dado sempre foi booleano; o tipo agora diz isso
<code>orders.order_id</code>	<code>smallint</code>	<code>integer</code>	o teto de 32 767 já estava a 1/3 do caminho
<code>region_description</code>	—	<code>region_name</code>	o nome antigo descrevia o tipo, não o significado

3. Normalização, com exemplo do próprio Northwind

As três formas normais, cada uma com o caso concreto desta base.

1FN — cada campo guarda um valor só

Onde a base cumpre. Os produtos vendidos não estão em `produto_1`, `produto_2`, `produto_3` dentro do pedido: estão em `order_items`, uma linha por produto. Se estivessem em colunas repetidas, seria preciso fixar um teto arbitrário, e o pedido com um produto a mais quebraria o modelo.

Onde a base NÃO cumpre — e eu deixei assim, de propósito. `products.quantity_per_unit` guarda coisas como "10 boxes x 20 bags". São quatro informações espremidas num texto: quantidade, embalagem externa, quantidade interna e unidade. Não é atômico, logo viola a 1FN.

Mantive porque separar exigiria **interpretar 77 textos heterogêneos** e chutar o significado de vários deles — e porque nenhuma das 16 perguntas de negócio usa esse campo. É uma violação **declarada**, não uma que passou despercebida.

2FN — nada depende de só um pedaço da chave composta

A tabela que importa aqui é `order_items`, cuja chave é o par (`order_id`, `product_id`).

O caso interessante é `unit_price`. Se ela significasse "o preço do produto", dependeria apenas de `product_id` — metade da chave — e seria violação de 2FN. Mas ela significa **o preço praticado naquela venda**, que depende do pedido e do produto. Depende da chave inteira. Está correto.

A prova de que os dois preços são coisas diferentes são os mesmos **662 itens (30,7%)** de `docs/01` §4.4: o preço da venda difere do catálogo atual.

O contra-exemplo ajuda a fixar: se eu tivesse guardado `product_name` dentro de `order_items`, aí sim seria violação — o nome do produto depende só de `product_id`.

3FN — nenhum campo comum determina outro campo comum

Se `orders` guardasse `customer_company_name`, haveria uma cadeia `order_id -> customer_id -> company_name`: dependência transitiva, violação de 3FN. Bastaria o cliente mudar de razão social para os pedidos antigos passarem a mentir. Por isso `orders` guarda **só** `customer_id`.

A pergunta difícil: e as seis colunas `ship_*` do pedido, que em 90% dos casos são cópia literal do endereço do cliente?

Não violam a 3FN — e o motivo é preciso. Dependência funcional não é "os valores costumam coincidir", é "conhecer A **determina** B". Conhecer o cliente **não** determina o endereço de entrega daquele pedido, e o número depende de quantas colunas se compara:

Critério	Pedidos iguais ao cadastro	Diferentes
só <code>ship_address</code> , <code>ship_city</code> , <code>ship_country</code>	782 (94,2%)	48 (5,8%)
as seis colunas <code>ship_*</code>	748 (90,1%)	82 (9,9%)

Pelo critério estrito — as seis colunas, que é o que "o endereço inteiro" significa — são **82 pedidos, quase 10%**, que saíram para lugar diferente do cadastro. Em qualquer um dos dois critérios a conclusão é a mesma: o endereço de entrega é atributo do pedido, não do cliente.

O que existe ali é outra coisa, e é honesto declarar: **falta uma entidade**. O modelo ideal teria os endereços do cliente como entidade própria, e o pedido referenciaria qual foi usado. Não implementei porque exigiria inventar identificadores de endereço e decidir o que fazer com os 82 divergentes — e porque a decisão não muda nenhuma das 16 perguntas. Fica registrada como dívida consciente.

E quanto à BCNF? As 11 tabelas também estão, mas a afirmação merece a análise em vez do parêntese — porque o contraexemplo clássico se sustenta nos dados desta base. `postal_code` determina `city` e `country` em **100% das linhas**: são 86 CEPs distintos entre os 90 clientes com CEP, e **zero** casos de um mesmo CEP em cidades diferentes. Se isso fosse dependência funcional, `customers` não estaria em BCNF, porque `postal_code` não é chave candidata.

Mas não é — e o critério é o mesmo que resolveu o caso do endereço de entrega logo acima: **dependência funcional é uma regra do domínio, não uma regularidade da amostra**. CEP é reorganizado, cidade é desmembrada, e esta base não é autoridade de endereçamento postal. Coincidir em 100% de 90 linhas não é determinar.

3.1 Duas cardinalidades que a chave estrangeira não consegue expressar

Os relacionamentos de `docs/02` viraram 11 chaves estrangeiras. Mas chave estrangeira só garante um lado: que **o filho aponta para um pai que existe**. Ela não garante o outro, a **participação mínima** — que o pai tenha pelo menos um filho.

São **11 chaves estrangeiras** para os 10 relacionamentos de `docs/02`, pelo motivo já explicado lá: o N:N de ATUAÇÃO conta duas vezes. E duas cardinalidades do diagrama caem no buraco da participação mínima:

- `PEDIDO contém ITEM DO PEDIDO (1,N)` — "pedido sem item não é venda, é formulário em branco". O `nw` aceitaria um `INSERT` em `orders` sem nenhum item.
- `REGIÃO agrupa TERRITÓRIO (1,N)` — "região sem território não existe como região".

Garantir isso no banco exigiria *trigger* de constraint postergada, verificando no fim da transação. Não implementei: é peso desproporcional para o ganho, e uma trigger mal escrita quebra a carga em lote. **Ficam declaradas como regra de aplicação** — e essa é uma pergunta clássica de banca, que agora tem resposta pronta em vez de improviso.

O critério que decidiu os três casos acima, em uma frase: normalizei tudo que tinha dependência funcional real; deixei como está o que só parecia redundante. Normalizar até doer é tão errado quanto não normalizar.

4. Índices: quatro entraram, seis foram recusados

Foram dez candidatos. Nenhum entrou sem `EXPLAIN` provando que o planejador o escolhe.

Índice	Serve às perguntas	Plano medido
<code>orders (customer_id)</code>	06, 10, 11	Bitmap Index Scan
<code>orders (employee_id)</code>	04, 15	Bitmap Index Scan
<code>orders (order_date)</code>	02, 08, 09, 16	Index Only Scan
<code>order_items (product_id)</code>	01, 07, 12	Bitmap Index Scan

O que explica quase todas as recusas: a base é minúscula. A maior tabela, `orders`, ocupa **15 páginas de 8 kB**. Ler 15 páginas inteiras é barato, então o planejador só troca a varredura por índice quando o filtro é bem seletivo.

Os seis recusados, por motivo:

- `order_items (order_id)` — redundante. A PK é `(order_id, product_id)` e começa por `order_id`, então ela mesma atende.

Aqui cabe uma precisão que vale ponto na banca: **não** é verdade que um índice composto "só serve" para a busca que começa pela primeira coluna. Pela segunda ele continua utilizável — o PostgreSQL varre o índice inteiro. O que acontece é que isso costuma sair mais caro que ler a tabela, e por isso o planejador prefere o Seq Scan. Comprovado: removendo `order_items_product_idx` e desligando `enable_seqscan`, o banco usa a PK com `Index Cond` em `product_id`. A formulação correta é **"a PK cobre mal essa busca"**, não "não cobre" — e é isso que justifica o índice dedicado.

- `orders (customer_id, order_date)` — composto, criado para o caso clássico "últimos pedidos do cliente X". Ele **tem** um nicho, mas estreito, e a medição mostra exatamente onde:

Consulta	Plano		---	---	cliente com 30 pedidos, `LIMIT 5`	lê `orders_date_idx` ao contrário — ignora o composto
cliente com 6 pedidos, `LIMIT 5`	usa `orders_customer_idx` e ordena depois — ignora o composto					
cliente com 6 pedidos, `LIMIT 1`	**usa o composto**					

Ou seja: ele só ganha quando o filtro é muito seletivo e a consulta pede uma linha só. **Nenhuma das 16 perguntas de negócio tem essa forma**, então o custo de manter não se paga. Recusado por não servir a este projeto — não por ser inútil em geral.

- `products (category_id)`, `territories (region_id)`, `employee_territories (territory_id)` — essas tabelas ocupam **uma** página. Ler a página custa um acesso; ler o índice e depois a tabela custa dois.
- `products (supplier_id)` — dois motivos, e o primeiro é o interessante: a constraint **UNIQUE (supplier_id, product_name)** **já cria um índice**, e `supplier_id` é a primeira coluna dele, então outro índice seria duplicata. *Toda constraint UNIQUE vem com um índice embutido* — vale conferir o que já se tem antes de criar mais um. O segundo motivo é o de sempre: com a tabela em uma página, o planejador nem esse índice de brinde usa.

Em produção, com milhões de pedidos, vários desses passariam a valer a pena. A decisão vale para **este** volume, e está registrada assim de propósito.

5. Quatro views, nenhuma decorativa

View	Responde	Serve às perguntas
<code>vw_venda_item</code>	uma linha por produto vendido, com receita calculada	01, 04, 07, 10, 11, 14, 16
<code>vw_pedido_resumo</code>	uma linha por pedido: valor, itens e prazos	02, 06, 13
<code>vw_estoque_critico</code>	o que está abaixo do ponto de reposição	05
<code>vw_hierarquia_funcionarios</code>	a árvore da equipe, por CTE recursiva	15

Elas existem porque o cálculo de receita — `preço × quantidade × (1 - desconto)` — apareceria em 11 das 16 consultas, e cálculo repetido é cálculo que uma hora sai diferente em algum lugar.

`vw_pedido_resumo` usa `LEFT JOIN` com `order_items`, e isso é consequência direta da §3.1: como o banco aceita um pedido sem itens, uma junção interna o faria desaparecer da view em silêncio — e o "uma linha por pedido" viraria mentira justamente no caso anômalo que se quer enxergar. Com `LEFT`, o pedido vazio aparece com 0 itens e R\$ 0,00.

Um detalhe que a banca pode cobrar. A receita total pela view dá **1.265.793,29**, e a soma sem arredondar dá **1.265.793,04**. A diferença de R\$ 0,25 é proposital: a view arredonda **cada item**, como faz uma nota fiscal, onde toda linha tem um valor em centavos. Arredondar só no fim daria um total que não bate com a soma das linhas impressas.

6. Reproduzir do zero

```
docker compose up -d
for f in 00_northwind_original 10_ddl 20_load 21_validacao_carga 30_indexes 40_views 50_evidencias; do
  docker compose exec -T postgres psql -U pi -d northwind -v ON_ERROR_STOP=1 -f /sql/$f.sql
done
```

O laço começa pelo dump: num clone novo o `docker compose up -d` já o carrega sozinho (está montado em `docker-entrypoint-initdb.d`), mas em quem já tinha o volume criado ele precisa rodar, senão a carga do `nw` não encontra o `public`.

O `-v ON_ERROR_STOP=1` não é enfeite: sem ele o `psql` continua depois de um erro e ainda sai com código 0 — ou seja, a carga "passaria" com uma constraint violada no meio. É essa flag que sustenta a afirmação da §2.

Os scripts são idempotentes: `10_ddl.sql` recria o schema do zero e `20_load.sql` trunca antes de inserir. Rodar duas vezes dá o mesmo resultado.

Apêndice A — Validação da carga public → nw

Saída bruta de `sql/21_validacao_carga.sql`, executada contra o banco. É a prova de que a migração para o schema do projeto não perdeu linha, centavo nem coluna. As outras treze evidências do trabalho estão no repositório, em `apresentacao/evidencias/`.

```
=== 1. Contagem de linhas: origem vs destino ===
(o total e 3311, nao os 3362 do docs/01: a diferenca sao as 51 linhas de
us_states, tabela que ficou fora do modelo por decisao declarada)
      tabela      | origem | destino | divergencia
-----+-----+-----+-----
regions           |      4 |       4 |          0
territories       |     53 |      53 |          0
categories        |      8 |       8 |          0
suppliers         |     29 |      29 |          0
shippers          |      6 |       6 |          0
customers         |     91 |      91 |          0
employees         |      9 |       9 |          0
employee_territories |    49 |      49 |          0
products          |     77 |      77 |          0
orders            |    830 |     830 |          0
order_items       |   2155 |    2155 |          0
TOTAL             |   3311 |    3311 |          0
(12 rows)
```

```
=== 2. O dinheiro: receita total nos dois schemas ===
(a diferenca NAO e erro de carga: e o erro de arredondamento do real, que o numeric elimina)
      receita_origem_real | receita_destino_numeric | origem_convertida_numeric
-----+-----+-----
1265793.038653364 |      1265793.03950 |      1265793.03950
(1 row)
```

```
=== 3. Somas de controle por coluna monetaria ===
      coluna      | origem | destino | divergencia
-----+-----+-----+-----
orders.freight    | 64942.69 | 64942.69 |      0.00
products.unit_price | 2220.21 | 2220.21 |      0.00
order_items.unit_price | 56500.91 | 56500.91 |      0.00
order_items.quantity |   51317 |   51317 |          0
order_items.discount |  121.040 |  121.040 |     0.000
(5 rows)
```

```
=== 4. As conversoes de tipo preservaram o significado? ===
      verificacao      | origem | destino
-----+-----+-----
discontinued: 1 na origem = true no destino |    10 |    10
shipped_date nulo (pedidos nao enviados) |    21 |    21
reports_to nulo (topo da hierarquia) |     1 |     1
faixa de datas identica (dias entre 1o e ultimo pedido) |   671 |   671
(4 rows)
```

```
=== 5. Nenhuma chave se perdeu no caminho (anti-join nos dois sentidos) ===
      verificacao      | linhas
-----+-----
pedidos na origem sem correspondente no nw |      0
itens na origem sem correspondente no nw |      0
clientes na origem sem correspondente no nw |      0
(3 rows)
```

```

=== 6. 0 que o schema nw ganhou em regras de negocio ===
(PK e FK caem de 14/13 para 11/11 porque 3 tabelas ficaram fora do modelo:
as 2 FKs a menos sao as de customer_customer_demo, que esta vazia.
0 ganho esta nas linhas CHECK, de 0 para 17, e UNIQUE, de 0 para 4.)

```

tipo	public_origem	nw_projeto
CHECK	0	17
FOREIGN KEY	13	11
PRIMARY KEY	14	11
UNIQUE	0	4

(4 rows)

```

=== 7. Colunas que deixaram de aceitar nulo ===
colunas_agora_obrigatorias

```

19

(1 row)

```

=== 8. Nenhuma coluna se perdeu sem declaracao ===
(a carga descarta 3 colunas de proposito, listadas no cabecalho de 20_load.sql.
Qualquer outra coluna da origem sem correspondente no nw e PERDA SILENCIOSA.)

```

coluna_da_origem	situacao
categories.picture	descartada por decisao (declarada)
employees.photo	descartada por decisao (declarada)
employees.photo_path	descartada por decisao (declarada)

(3 rows)

```

=== 9. O schema opera? (as chaves sinteticas geram valor) ===
(carregar nao basta: um modelo que nao aceita uma venda nova nao e um modelo)

```

tabela	sequence_no_maior_id_usado	geracao
categories	8	GENERATED BY DEFAULT
employees	9	GENERATED BY DEFAULT
orders	11077	GENERATED BY DEFAULT
products	77	GENERATED BY DEFAULT
regions	4	GENERATED BY DEFAULT
shippers	6	GENERATED BY DEFAULT
suppliers	29	GENERATED BY DEFAULT

(7 rows)